



UNIVERSIDADE ESTADUAL DO PIAUÍ - UESPI
CAMPUS DR^a JOSEFINA DEMES
CURSO BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: PADRÕES DE PROJETO



Estudo de Caso 5 — Singleton

Valor: 2,0 pontos

Sistema de Controle de Impressão

Uma empresa possui vários computadores conectados a uma **impressora central**.

Diversas partes do sistema podem solicitar impressões:

- setor financeiro;
- setor administrativo;
- setor de recursos humanos;
- setor de atendimento.

Todas as solicitações devem ser encaminhadas para **o mesmo gerenciador de impressão**, responsável por controlar a fila de documentos.

O sistema atualmente permite que cada setor crie seu próprio objeto GerenciadorImpressao.

Isso pode gerar problemas, pois cada objeto poderia manter uma fila diferente:

Gerenciador A → fila: documento1, documento2

Gerenciador B → fila: documento3

Gerenciador C → fila: documento4, documento5

Nesse cenário, o sistema não possui uma fila centralizada.



Problema

O sistema precisa garantir que exista **apenas uma instância de GerenciadorImpressao durante a execução da aplicação**.

Todos os setores devem utilizar essa mesma instância para adicionar documentos à fila.

A solução deverá utilizar o **padrão Singleton**.



O que os alunos devem fazer

1. Criar a classe GerenciadorImpressao

A classe deverá possuir uma estrutura adequada para implementar o Singleton.

Ela deve controlar a própria criação e impedir que outras classes criem diretamente novas instâncias.

2. Implementar a instância única

Crie um mecanismo para garantir que:

- a primeira solicitação crie o gerenciador;
- as solicitações seguintes retornem a mesma instância;
- não seja possível criar o gerenciador utilizando new a partir de outras classes.

3. Criar o gerenciamento da fila

O GerenciadorImpressao deverá possuir uma coleção para armazenar os documentos.

Cada documento pode ser representado apenas pelo seu nome:

"Relatorio_Financeiro.pdf"

"Folha_de_Pagamento.pdf"

"Contrato_Cliente.pdf"

Crie um método para adicionar documentos à fila.

Por exemplo:

adicionarDocumento("Relatorio_Financeiro.pdf")

4. Criar um método para visualizar a fila

Implemente um método que apresente os documentos atualmente aguardando impressão.

Por exemplo:

Fila de impressão:

1. Relatorio_Financeiro.pdf

2. Folha_de_Pagamento.pdf

3. Contrato_Cliente.pdf

5. Criar o programa principal

No main, simule diferentes setores utilizando o sistema.

Por exemplo:

Setor Financeiro

→ adiciona "Relatorio_Financeiro.pdf"

Setor RH

→ adiciona "Folha_de_Pagamento.pdf"

Setor Administrativo

→ adiciona "Contrato_Fornecedor.pdf"

Todos devem obter o gerenciador através do mecanismo criado por vocês.

Não deve ser possível fazer:

```
new GerenciadorImpressao();
```

fora da própria classe.

6. Comprove que é Singleton

O programa deverá criar pelo menos duas referências:

```
gerenciador1
```

```
gerenciador2
```

obtidas em momentos diferentes.

Depois, demonstre que elas representam **a mesma instância**.

Por exemplo, utilizando:

```
gerenciador1 == gerenciador2
```

O resultado esperado é:

```
true
```



Exemplo do comportamento esperado

A execução poderá produzir algo semelhante a:

```
=== SISTEMA DE IMPRESSÃO ===
```

Setor Financeiro:

Documento adicionado: Relatorio_Financeiro.pdf

Setor RH:

Documento adicionado: Folha_de_Pagamento.pdf

Setor Administrativo:

Documento adicionado: Contrato_Fornecedor.pdf

=== FILA DE IMPRESSÃO ===

1. Relatorio_Financeiro.pdf

2. Folha_de_Pagamento.pdf

3. Contrato_Fornecedor.pdf

As referências apontam para a mesma instância?

true



Requisitos obrigatórios

O programa deve:

- implementar o padrão **Singleton**;
- possuir construtor privado;
- manter uma única instância da classe;
- fornecer um método de acesso à instância;
- possuir uma fila de documentos;
- permitir adicionar documentos;
- permitir visualizar a fila;
- demonstrar que duas referências obtidas em momentos diferentes apontam para a mesma instância;
- impedir a criação direta de GerenciadorImpressao fora da classe.



Desafio extra

Depois de implementar a solução, responda:

Por que seria um problema permitir que cada setor criasse seu próprio GerenciadorImpressao?

E:

Qual característica do Singleton garante que todos os setores trabalhem com a mesma fila?

Essa parte é interessante porque obriga o aluno a **explicar o padrão**, e não apenas fazer o código funcionar.